UNIVERSITY of BRADFORD

Files and exceptions

Software Development COS7009-B

What is a File?

- A file is a collection of data that is stored on secondary storage like a disk drive
- Accessing a file means establishing a connection between the file and the program and moving data between the two

Making a File Object

- my_file = open("my_file.txt", "r")
- my_file is the file object. It contains the buffer of information. The open function creates the connection between the disk file and the file object. The first quoted string is the file name on disk, the second is the mode to open it (here, "r" means to read)

File paths

- **absolute path**: specifies a drive or a top `"/"` folder

  `C:/Documents/smith/hw6/input/data.csv`

  – Windows can also use backslashes to separate folders.

- **relative path**: does not specify any top-level folder

  `names.dat`
  `input/kinglear.txt`

  – Assumed to be relative to the *current directory*:

  ```
  my_file= open("data/readme.txt","r")
  ```

  If our program is in `H:/hw6`,
  `open` will look for `H:/hw6/data/readme.txt`

Different modes

Mode	How opened	File Exists	File Does Not Exist
'r'	read-only	Opens that file	Error
'w'	write-only	Clears the file contents	Creates and opens a new file
'a'	write-only	File contents left intact and new data appended at file's end	Creates and opens a new file
'r+'	read and write	Reads and overwrites from the file's beginning	Error
'w+'	read and write	Clears the file contents	Creates and opens a new file
'a+'	read and write	File contents left intact and read and write at file's end	Creates and opens a new file

Careful with Write Modes

- Be careful if you open a file with the ‘w’ mode. It sets an existing file’s contents to be empty, destroying any existing data.
- The ‘a’ mode is nicer, allowing you to write to the end of an existing file without changing the existing contents

Text Files Use Strings

- If you are interacting with text files, remember that everything is a string
  - Everything read is a string
  - If you write to a file, you can only write a string

Close

- When the program is finished with a file, we close the file
- Flush the buffer contents from the computer to the file
- Tear down the connection to the file
- Close is a method of a file obj
  - file_obj.close()
- All files should be closed!

.read() method

Line 1
Line 2

- read the whole file into a single string
- Allow you to read a specified number of characters from a file

```
>>> text_file = open("read_it.txt", "r")
>>> print(text_file.read(1))
L
>>> print(text_file.read(5))
??
```

.read() method

- Each subsequent read() begins where the last ended
- return the empty string when reaches the end
- To go back at the beginning of the file

```
>>> text_file.close()
>>> text_file = open("read_it.txt", "r")
```

.readline() method

Line 1
Line 2

- Returns the current line as a string

```
>>> text_file = open("read_it.txt", "r")
>>> print(text_file.readline())
Line 1

>>> print(text_file.readline())
Line 2
```

Readline()

Line 1
Line 2

- Also let you read characters from the current line

```
>>> text_file = open("read_it.txt", "r")
>>> print(text_file.readline(1))
L
>>> print(text_file.readline(5))
ine 1
```

readlines()

Line 1
Line 2

- Reads a text file into a list
- Each line of the file becomes a string element in the list

```
>>> text_file = open("read_it.txt", "r")
>>> lines = text_file.readlines()
>>> lines
['Line 1\n', 'Line 2']
```

Looping through a file

- Direct though the lines of a file

```
>>> text_file = open("read_it.txt", "r")
>>> for line in text_file:
        print(line)
```

Writing to a File

- Once you have created a file object, opened for reading, you can use the print command
- You add file=file to the print command

```
# open file for writing:
#     creates file if it does not exist
#     overwrites file if it exists
>>> temp_file = open("temp.txt","w")
>>> print("first line", file=temp_file)
>>> print("second line", file=temp_file)
>>> temp_file.close()
```

.write() method

- Write a string to the file
- \n is needed

```
>>>temp_file = open(“temp.txt”,’w’)
>>>temp_file.write(“first line\n")
>>>temp_file.write(“second line 2\n")
```

.writelines()

- Write a list of strings to a file

```
>>>temp_file = open(“temp.txt”,’w’)
>>>lines = [“first line 2\n“,“second line 2\n"]
>>>temp_file.writelines(lines)
```

File methods

Method	Description
`close()`	Closes the file. A closed file cannot be read from or written to until opened again.
`read([size])`	Reads *`size`* characters from a file and returns them as a string. If size is not specified, the method returns all of the characters from the current position to the end of the file.
`readline([size])`	Reads *`size`* characters from the current line in a file and returns them as a string. If size is not specified, the method returns all of the characters from the current position to the end of the line.
`readlines()`	Reads all of the lines in a file and returns them as elements in a list.
`write(output)`	Writes the string *`output`* to a file.
`writelines(output)`	Writes the strings in the list *`output`* to a file.

Exceptions

How to Deal with Problems

- Most modern languages provide methods to deal with ‘exceptional’ situations
- Gives the programmer the option to keep the user from having the program stop without warning
- In general, we have assumed that the input we receive is correct (from a file, from the user).
- This is almost never true. There is always the chance that the input could be wrong
- Our programs should be able to handle this.

Error Names

Errors have specific names, and Python shows them to us all the time.

```
>>> input_file = open("no_such_file.txt", 'r')
Traceback (most recent call last):
  File "<pyshell#0>", line 1, in <module>
    input_file = open("no_such_file.txt", 'r')
IOError: [Errno 2] No such file or directory: 'no_such_file.txt'
>>> my_int = int('a string')
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    my_int = int('a string')
ValueError: invalid literal for int() with base 10: 'a string'
>>>
```

You can recreate an error to find the correct name. Spelling counts!

try Suite

- the `try` suite contains code that we want to monitor for errors during its execution.
- if an error occurs anywhere in that `try` suite, Python looks for a handler that can deal with the error.
- if no special handler exists, Python handles it, meaning the program halts and with an error message as we have seen so many times ☹

`except` Suite

- An `except` suite (perhaps multiple `except` suites) is associated with a `try` suite.
- Each exception names a type of exception it is monitoring for.
- If the error that occurs in the `try` suite matches the type of exception, then that `except` suite is activated.

`try/except` Group

- If no exception in the `try` suite, skip all the `try/except` to the next line of code
- If an error occurs in a `try` suite, look for the right exception
- If found, run that `except` suite and then skip past the `try/except` group to the next line of code
- If no exception handling found, give the error to python

Example

```
try:
      num = float(input("Enter a number: "))
except:
      print("Something went wrong!")
```

- IF the call to float() raises an exception, the user is informed
- If no exception is raised, the program skips the except clause

Specifying an exception type

Exception Type	Description
`IOError`	Raised when an I/O operation fails, such as when an attempt is made to open a nonexistent file in read mode.
`IndexError`	Raised when a sequence is indexed with a number of a nonexistent element.
`KeyError`	Raised when a dictionary key is not found.
`NameError`	Raised when a name (of a variable or function, for example) is not found.
`SyntaxError`	Raised when a syntax error is encountered.
`TypeError`	Raised when a built-in operation or function is applied to an object of inappropriate type.
`ValueError`	Raised when a built-in operation or function receives an argument that has the right type but an inappropriate value.
`ZeroDivisionError`	Raised when the second argument of a division or modulo operation is zero.

Multiple exception types

- A single piece of code can result in different types of exceptions

```
>>> float('a')
Traceback (most recent call last):
  File "<pyshell#0>", line 1, in <module>
    float('a')
ValueError: could not convert string to float: 'a'
>>> float([1])
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    float([1])
TypeError: float() argument must be a string or a number, not 'list'
```

Multiple exception types

- In a single except clause

```
For value in ([1],'a'):
    try:
        print(float(value))
    except (TypeError,ValueError):
        print("Something went wrong!")
```

Multiple exception types

- Or in multiple except clauses

```
For value in ([1],'a'):
    try:
        print(float(value))
    except TypeError:
        print("shouldn't be a list!")
    except ValueError:
        print("should be a number!")
```

Reference

- Python Programming by Michael Dawson
- https://legacy.python.org/doc/essays/ppt/
- The practice of computing using Python by Punch & Enbody